GraphSentinel: A Trustworthy Temporal Graph Neural Network Model for Autonomous Lateral Movement Detection and Prevention

Thillai Nathan B
UG Student
Department of Artificial Intelligence and Machine Learning
Rajalakshmi Engineering College, Thandalam, Chennai
thillainathanb6@gmail.com

Vijay Anand J
UG Student
Department of Artificial Intelligence and Machine Learning
Rajalakshmi Engineering College, Thandalam, Chennai
231501181@rajalakshmi.edu.in

Nithish Balaji
UG Student
Department of Artificial Intelligence and Machine Learning
Rajalakshmi Engineering College, Thandalam, Chennai
231501115@rajalakshmi.edu.in

Anitha R
Assistant Professor
Department of Artificial Intelligence and Machine Learning
Rajalakshmi Engineering College, Thandalam, Chennai
anitha.r@rajalakshmi.edu.in

***Abstract*—Attackers who steal valid credentials can move between hosts through ordinary remote logons, which signature-based controls rarely stop. We present GraphSentinel, a detection-and-prevention model for such lateral movement that works directly on authentication logs. At its core, a temporal graph neu- ral network (TGN) keeps a memory for every account and host and scores each logon from 27 strictly causal features. The model fuses this score with rule-based evidence through a noisy-OR gate and a same-account chain rule, allows autonomous action only above a gate derived from a Wilson lower bound on preci- sion, and runs a bounded response loop that forces re-authenti- cation, verifies whether the account keeps moving, escalates to a reversible two-hour lock, and reverts. A locally hosted language model writes analyst reports in which every claim must cite rec- orded evidence. On the Los Alamos corpus the TGN reaches a test PR-AUC of 0.821, but the figure rests on one attacker host and drops to 0.002 without it; on a corrected corpus a logistic regression over the same features scores 0.956, above every TGN-based detector (at most 0.676). In a replay of 100 simulated campaigns the model acts autonomously on 38 and prevents 14.6% of attacker hops (27.8% for single-credential chains) at 5.5 automatic actions per 10,000 benign logons, and escalation keeps 9.3% prevention even when session resets always fail. On 21 public Windows attack recordings it ranks attacks above chance (ROC-AUC 0.66) but alerts on only two, so thresholds must be re-derived for each network.**

Keywords—lateral movement, temporal graph neural networks, authentication logs, MITRE ATT&CK, automated incident response, trustworthy AI, security operations

I. INTRODUCTION

Serious intrusions rarely end on the first machine an attacker controls. After gaining a foothold, the attacker harvests credentials and uses them to reach more valuable systems, the tactic that MITRE ATT&CK calls lateral movement (TA0008) [1]. Its common techniques, such as remote ser- vices over SMB, WinRM or RDP (T1021), valid domain ac- counts (T1078) and pass-the-hash or pass-the-ticket (T1550), all leave the same footprint: an authentication event on a do- main controller or on the destination host. Each event looks legitimate on its own, because the credential is genuine. What gives the attacker away is the pattern, an account that suddenly reaches hosts it has never used, often from a host it reached a few minutes earlier.

Graph-based detection has made steady progress on this problem, from mining login patterns [2] and reconstructing login paths [3] to graph embeddings [4], [5] and temporal link prediction with graph neural networks [6], [7]. Most of these systems are evaluated on the Los Alamos National Laboratory (LANL) authentication corpus [8], with several studies reporting very high scores. Two practical gaps remain. First, detection usually ends with a ranked list of alerts, while scripted lateral movement can complete several hops before an analyst opens the first one, so the attacker's dwell time depends on how quickly a detection turns into containment. Second, recent re-evaluations show that LANL results depend heavily on preprocessing and labelling choices [9], in line with long-standing concerns about how machine learning is evaluated in security [10], [11].

This paper presents GraphSentinel, a trustworthy detection-and-prevention model built around both gaps. We use the word trustworthy in a narrow and testable sense: the model scores events only from the past, acts on its own only where a statistical precision bound allows it, keeps every automatic action bounded, reversible and logged, explains itself only through recorded facts, and is evaluated with its failure cases reported. The objectives of the work are:

1. To ingest heterogeneous authentication logs (LANL, Windows Security, SSH, Zeek, cloud sign-ins, CSV or JSON) into one canonical event stream with automatic format detection.
2. To score every logon with a temporal graph neural network [12] that keeps a memory for each account and host and uses 27 features computed strictly from past events.
3. To fuse the learned score with explicit evidence in an auditable way, and to derive the threshold for autonomous action from a lower confidence bound on precision.
4. To contain lateral movement automatically within explicit limits (contain, verify, escalate, revert) under

hourly and per-account budgets, with every action recorded.

5) To produce grounded triage and incident reports with a locally hosted language model.
6) To measure prevention, false-action cost and generalisation, including the cases in which the learned component does not help.

The evaluation also produced three findings that we consider as important as the system itself: the headline LANL score is dominated by one attacking host; once local logons are removed, a linear model over the same features outperforms the TGN; and a model that ranks attacks sensibly on an unseen network can still miss almost all of them at a threshold calibrated elsewhere. Sections II to VII cover related work, the threat model, the architecture, the model, the results and our conclusions.

II. RELATED WORK

Siadati and Memon [2] model the normal login structure of an enterprise with market-basket pattern mining and flag logins that break it, detecting 82% of malicious logins at a 0.3% false-positive rate on five months of data from a financial firm. Hopper [3] builds a graph of logins between internal machines, infers the causal paths that link them, and alerts on paths that switch credentials or reach unusual destinations; on fifteen months of enterprise logs it detects 94.5% of more than 300 realistic attack scenarios with fewer than nine alerts a day. Both works show that the path, not the single login, carries the signal, an idea that GraphSentinel's chain rule applies online.

Log2vec [4] turns audit records into a heterogeneous graph with rule-defined relations and separates malicious activity by clustering random-walk embeddings. Bowman et al. [5] learn unsupervised embeddings of authenticating entities and score each authentication with a link predictor. Euler [6] stacks a graph neural network on a recurrent encoder over graph snapshots and distributes it across machines, and Jbeil [7] applies inductive temporal-graph learning to LANL authentications, reporting an AUC above 0.99 even when part of the graph is unseen in training. These detectors build on continuous-time graph models such as JODIE [13], TGAT [14] and TGN [12], which update a state per node as timestamped interactions arrive; GraphSentinel uses a TGN-style memory with a gated recurrent unit (GRU) [15].

Benchmarks for dynamic graphs can reward memorisation: a baseline that only remembers past edges does well under common protocols [16], and a simple MLP-based model matches more complex temporal architectures [17]. In security, Sommer and Paxson [10] explain why anomaly detection is hard to evaluate, Arp et al. [11] list pitfalls such as sampling bias, spurious correlations and weak baselines, and Larroche [9] re-evaluates three graph-based lateral-movement detectors on LANL and OpTC under standardised preprocessing and labelling and finds results well below those first reported.

On the response side, NIST's incident-response profile for CSF 2.0 treats containment of an incident as a core outcome of the Respond function [18], and surveys of intrusion response systems stress that the cost of a wrong automatic response must be weighed against the damage it prevents [19]. Learned lateral-movement detectors, however, rarely specify how an alert becomes an action. Large language models are now used to summarise alerts, but their tendency to state unsupported content [20] makes unchecked output unsafe for security decisions. Table I compares representative work.

TABLE I. COMPARATIVE ANALYSIS OF RELATED WORK ON LATERAL-MOVEMENT DETECTION

Author & Year	Method	Data	Contribution	Limitation
Siadati & Memon (2017) [2]	Market-basket mining of login patterns	Enterprise logins, 5 months	82% of malicious logins at 0.3% false positives	Single-login view; alerts only
Liu et al. (2019) [4]	Heterogeneous graph embedding and clustering	CERT, LANL	Rule-defined relations between log entries	Offline batch; hand-built relations
Bowman et al. (2020) [5]	Unsupervised graph embedding, link prediction	Simulated network, LANL	Outperforms heuristic baselines	Static embeddings; alerts only
Ho et al. (2021) [3]	Login-path inference, credential switching	Enterprise logs, 15 months	94.5% detection, < 9 alerts per day	Depends on path inference; alerts only
King & Huang (2022) [6]	GNN with recurrent encoder over snapshots	LANL	Scalable temporal link prediction	Discrete snapshots; alerts only
Khoury et al. (2024) [7]	Inductive temporal graph learning	LANL	AUC > 0.99 with unseen nodes	Offline evaluation; alerts only
Larroche (2026) [9]	Standardised re-evaluation protocol	LANL, OpTC	Shows preprocessing changes reported results	Evaluation study, no detector
This work	TGN with noisy-OR fusion, chain rule, precision-gated autonomous response, grounded reports	LANL, OTRF	Measures prevention and false-action cost end to end	Prevention measured on simulated campaigns

III. THREAT MODEL AND PROBLEM FORMULATION

A. Adversary Model

The adversary already controls one workstation, the foothold h_0, and has obtained credentials on it, for example through OS credential dumping (T1003). From there it moves with valid accounts over remote services (T1021, T1078) or with stolen hashes and Kerberos tickets (T1550). It may rotate between harvested accounts, slow down to blend into normal traffic, and guess passwords before a hop succeeds. Every hop therefore produces at least one authentication event that reaches the defender. Out of scope are an adversary who can tamper with the log pipeline or the detection host, movement that never authenticates, and poisoning of the training data; these require controls outside this work.

B. Defender Capabilities and Trust Requirements

The defender collects authentication events from domain controllers and hosts in near real time, can end an account's sessions, revoke its cloud tokens and disable it, and has a security operations centre (SOC) analyst who approves disruptive actions. GraphSentinel complements zero-trust

architectures [21] by judging each logon in the context of the path it extends. Because an automated responder that errs can hurt users as much as an attacker, we require five properties: (T1) *causality*, a score uses only events before the one being scored; (T2) *calibrated autonomy*, the model acts alone only above a threshold with a lower confidence bound on precision; (T3) *bounded, reversible impact*, automatic actions are rate-limited, reversible and disabled until armed; (T4) *complete mediation*, every action passes through one coordinator that records the command and its undo; and (T5) *faithful explanation*, generated reports may cite only recorded facts. T3 and T4 apply the classic secure-design principles of fail-safe defaults and complete mediation [22].

C. Problem Formulation

Authentication events arrive as a stream $e_1, e_2, \ldots$ over a temporal multigraph whose nodes are accounts and hosts. A lateral-movement campaign of length L is a sequence of successful events with sources σ_i and destinations δ_i in which every hop starts from the foothold or from a host reached earlier in the campaign (1).

$$\sigma_i \in \{h_0, \delta_1, \ldots, \delta_{i-1}\}, \quad i = 1, \ldots, L \tag{1}$$

For every event, and using only information available at its time, the defender outputs a risk $\rho_k \in [0, 1]$ and an action from none, alert, contain and lock. We seek a policy π that prevents as many campaign hops as possible while keeping automatic actions on benign logons below a budget β per 10,000 (2), where η is the prevented fraction defined in (19), $\mathrm{FA}(\pi)$ counts automatic actions on benign events and N_b is the number of benign events. We do not solve (2) directly; Section V builds a precision-gated policy that approximates it and Section VI measures both of its terms.

$$\max_\pi \mathbb{E}[\eta(\pi)] \quad \text{s.t.} \quad 10^4 \frac{\mathrm{FA}(\pi)}{N_b} \le \beta \tag{2}$$

IV. SYSTEM ARCHITECTURE

A. Datasets

The primary corpus is the LANL comprehensive cybersecurity events dataset [8], which covers 58 consecutive days of de-identified activity by 12,425 users on 17,684 computers. Its authentication file holds 1,051,430,459 events, each with the time, the source and destination user and computer, the authentication type (e.g., Kerberos or NTLM), the logon type, the orientation and the outcome. A separate file lists 749 red-team compromise events involving 104 accounts and 301 destination hosts. These labels are highly concentrated: 701 of the 749 events originate from one source computer and the other 48 from three more (26, 19 and 3 events), which shapes every evaluation on this corpus.

Two processed corpora are used. The corpus behind the shipped model (C1) keeps every event that matches a red-team record and one in 512 of the others over the first 16 days (543,615 events, 649 attacks), split chronologically into training, validation and test partitions with 316, 207 and 126 attacks so that each holds complete campaigns. A corrected slice (C2) covers the first 900,000 s, removes the 95,025,843 local logons (source equal to destination; none is an attack), which make up 53.8% of the raw stream, and keeps one in 448 of the remaining events (182,815 events; 50, 165 and 101 attacks per partition). Stride sampling makes training feasible on a laptop GPU (NVIDIA RTX 3050, 6 GB), but it inflates the prevalence of attacks and truncates benign histories; Section VI-E quantifies the consequence for rules.

The external test set comes from the OTRF Security Datasets project [23]: 29 public recordings of Windows Security telemetry from a small lab domain, each capturing one lateral-movement technique run with Empire, Covenant, Mimikatz or PurpleSharp, among them PsExec-style service creation over SMB admin shares (T1021.002), DCOM (T1021.003), WinRM (T1021.006), WMI execution (T1047) and the Zerologon exploit (T1210). Ground truth comes from the attacker console transcripts published with each recording. Six recordings were excluded before scoring because their metadata name no target or describe a different lab, and two produced no authentication edge, leaving 21 recordings with 77 attack and 118 benign events.

B. Architecture

Fig. 1 shows the architecture, which runs as a single process over one ordered event stream. In SOC terms, GraphSentinel sits between the SIEM that collects logs and the EDR and identity tools that act: it consumes authentication events and emits alerts, reports and bounded response actions. Source adapters for LANL, Windows Security (event IDs 4624, 4625, 4768, 4769 and 4776), SSH, Zeek, Entra ID, Okta and tabular files map records to a canonical event, and a detector selects the adapter when the format is not declared. An entity dictionary, frozen with the checkpoint, maps names to integer identifiers and hashes unseen names into 4,096 user and 16,384 host out-of-vocabulary buckets. The feature engine computes 27 features per event, the TGN produces a probability, rule channels add explicit evidence, and the fusion layer produces one risk per event, which is compared with an alert threshold and a stricter execution gate.

Every action passes through a single response coordinator that enforces the budgets and runs the prevention loop, and an executor that records the exact command and its undo for every connector (directory service, endpoint agent, host firewall). Connectors stay in dry-run mode until an operator arms them. Feature state and TGN memories are persisted, so a restart resumes warm; on C2 the deployed service reproduced the offline PR-AUC to five decimal places (0.603796 against 0.603798) at 315 events/s. The system is implemented in Python with PyTorch, a FastAPI service and a browser console, and its 754 automated tests pass.

V. MODEL BUILDING

A. Canonical Events and Causal Features

Each authentication becomes an event (3) with account u_k, source and destination hosts s_k and d_k, time t_k, outcome o_k and a feature vector $\mathbf{x}_k \in \mathbb{R}^{27}$ computed by a function Φ of the history $\mathcal{H}$ strictly before t_k, so no feature can encode the future or the label (T1).

$$e_k = (u_k, s_k, d_k, t_k, o_k, \mathbf{x}_k), \qquad \mathbf{x}_k = \Phi\big(e_k \mid \mathcal{H}_{<t_k}\big) \tag{3}$$

The features cover the event itself (outcome, authentication and logon type, orientation, hour of day), recency (time since the account's and the pair's previous events), novelty (first-seen flags, pair rarity, destination novelty), windowed behaviour from 5 min to 24 h (distinct destinations, authentication and failure rates, failures before success, share of new destinations) and graph structure (source fan-out, inbound users of the destination, historical degrees).

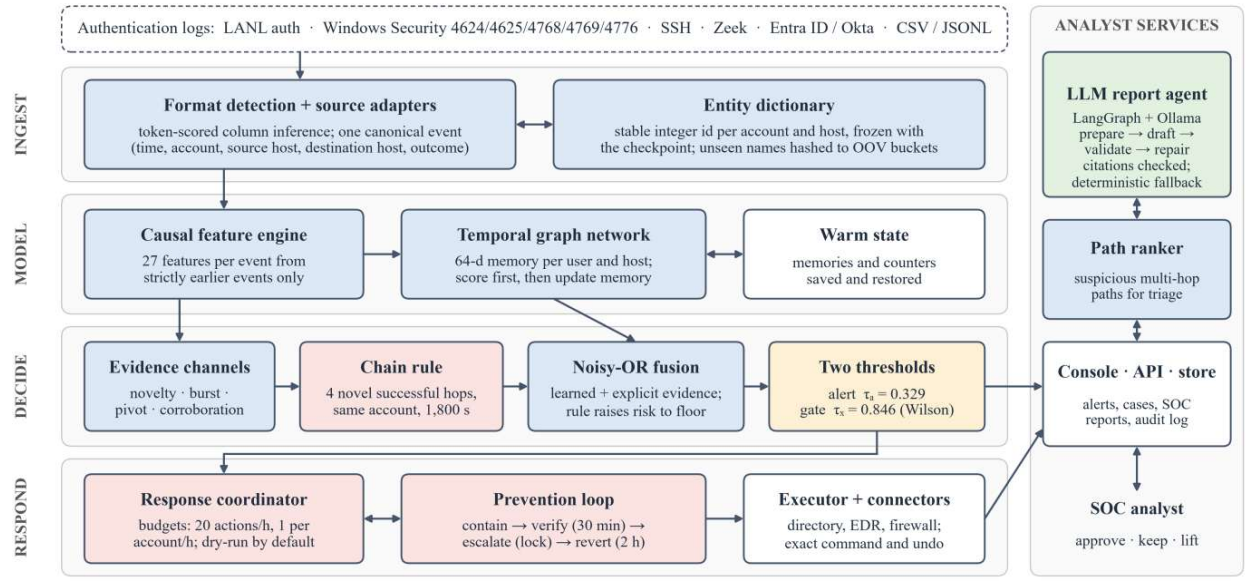


Fig. 1. Architecture of GraphSentinel. Logs of any supported format become one canonical event stream; each event is scored from past-only features and node memory, fused with explicit evidence, and compared with an alert threshold and a stricter execution gate that drives a bounded response loop.

B. Temporal Graph Network

Accounts and hosts are nodes with a 64-dimensional memory $\mathbf{m}$. The time since a node was last updated enters through a learnable harmonic encoding (4), in which $\boldsymbol{\omega}$ and $\boldsymbol{\varphi}$ are learned 16-dimensional frequencies and phases and the logarithm compresses gaps from seconds to weeks. For event e_k, the scorer concatenates the memories of the account and both hosts, the standardised features and the time encodings (5), and maps them to a probability through a two-layer network g with LayerNorm, GELU and dropout plus a linear residual projection $\mathbf{P}$ (6).

$$\psi(\Delta t) = \cos(\boldsymbol{\omega} \odot \log(1 + \Delta t) + \boldsymbol{\varphi}) \tag{4}$$

$$\mathbf{z}_k = [\mathbf{m}_u \| \mathbf{m}_s \| \mathbf{m}_d \| \hat{\mathbf{x}}_k \| \psi(\Delta t_u) \| \psi(\Delta t_d)] \tag{5}$$

$$p_k = \sigma(\mathbf{w}^\top (g(\mathbf{z}_k) + \mathbf{P}\mathbf{z}_k) + b) \tag{6}$$

Only after scoring does the event update memory (Fig. 2). A message function h forms one message per endpoint from its own memory, the other endpoint's memory, the features and the time encodings; messages that reach node i within a batch are weighted by attention (7), and their normalised sum updates the memory through a GRU cell (8). Scoring before updating guarantees that an event is judged by the state that existed before it; the implementation enforces this order and a regression test asserts it. Events are processed in 60-s buckets, a 20.7-fold training speed-up, and within a bucket memory is still read before any write.

$$\alpha_j = \mathrm{softmax}_j\left(\mathbf{a}^\top [\boldsymbol{\mu}_j \| \mathbf{m}_i]\right) \tag{7}$$

$$\mathbf{m}_i \leftarrow \mathrm{GRU}\left(\mathrm{LN}\left(\sum_j \alpha_j \boldsymbol{\mu}_j\right), \mathbf{m}_i\right) \tag{8}$$

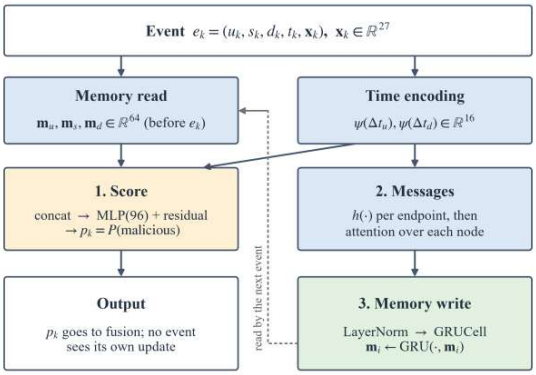


Fig. 2. TGN scorer. An event is scored from the memories that existed before it (step 1); only then are messages formed and aggregated (step 2) and memories written (step 3).

The model has 107,585 parameters. It is trained with the class-weighted binary cross-entropy (9), where β is the ratio of negative to positive training events capped at 120, using AdamW (learning rate 10^{-3}, weight decay 0.01), truncated back-propagation every 2,048 events and 16 epochs on C1; validation PR-AUC improved in every epoch, so early stopping (patience 5) never triggered.

$$\mathcal{L} = -\frac{1}{N}\sum_k [\beta y_k \log p_k + (1 - y_k)\log(1 - p_k)] \tag{9}$$

C. Evidence Fusion and Chain Rule

A weighted average would let the learned score veto every rule, because the weights of the other channels sum to less than the alert threshold. GraphSentinel instead fuses channels with a noisy-OR gate [24] (10), in which each channel c with signal $s_{k,c} \in [0, 1]$ has an independent reliability r_c. The reliabilities, fitted on validation data by coordinate ascent on PR-AUC, are 0.95 for the TGN, 0.35 for cross-source corroboration, 0.30 for burst and 0.05 for novelty and pivot. Any single strong channel can carry an alert, weak channels compound, and adding evidence never lowers the risk.

$$R_k = 1 - (1 - r_0 p_k) \prod_{c \in \mathcal{C}} \left(1 - r_c s_{k,c}\right) \quad (10)$$

The chain rule turns the path intuition of [3] into a streaming test (11). Let $\mathcal{R}_u$ be the set of hosts that account u has successfully reached before t_k, and n_u the number of such novel hops u made in the previous W = 1,800 s. An event is a chain hop when it succeeds, starts from a reached host, lands on a never-reached host, and follows at least three novel hops (χ_k = 1). Novelty counts only successful logons, because counting failed attempts would let an attacker who guesses a password before each hop make every hop look familiar. When the rule fires, the fused risk is raised to a floor (12), which in the autonomous policy equals the execution gate.

$$\chi_k = \mathbb{1}[o_k = 1 \ \wedge \ s_k \in \mathcal{R}_u \ \wedge \ d_k \notin \mathcal{R}_u \ \wedge \ n_u \geq 3] \quad (11)$$

$$\rho_k = \max(R_k, \tau_x \chi_k) \quad (12)$$

D. Execution Gate

Two thresholds are applied to ρ_k. The alert threshold τ_a = 0.329 was set on validation data at a budget of 25 false positives per 10,000 benign events (FP/10k). The execution gate τ_x (T2) is the lowest threshold for which every candidate above it keeps the Wilson lower bound [25] (13) on validation precision at 0.75 or more (14), considering only candidates with at least 20 alerts; $\hat{p}$ is the precision of the n alerts above a candidate and z = 1.96.

$$\mathrm{LB} = \frac{\hat{p} + \frac{z^2}{2n} - z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}} \quad (13)$$

$$\tau_x = \min\{\tau : \ \mathrm{LB}(\tau') \geq 0.75 \ \ \forall \tau' \geq \tau\} \quad (14)$$

On C1 this yields τ_x = 0.846, admitting 251 validation alerts at 80.5% precision (lower bound 75.1%). On the later test period the same gate yields 201 alerts at 53.2% precision [46.3%, 60.0%] and 8.1 benign actions per 10,000 events. The meaning of a fixed gate therefore decays with time, so both figures are stored with the constant and neither is quoted alone.

E. Bounded Response Loop

Fig. 3 shows the loop, which automates the containment outcome of the NIST Respond function [18] for credential-based movement. When $\rho_k \geq \tau_x$ and the budget (15) allows it (T3), where $\mathcal{D}(t-3600, t]$ is the set of automatic actions in the preceding hour and $t^{\mathrm{last}}{}_u$ is the time of the last action on account u, the coordinator contains the account by forcing re-authentication: the account is logged off the source host and its cloud refresh tokens are revoked. Below the gate or over budget, the plan waits for an analyst. Containment is then verified on the attacker's clock: if the account qualifies for an automatic action again within 1,800 s of its containment at $t^c{}_u$ (16), the reset has failed, and the loop escalates to an account lock, which is exempt from the per-account condition.

$$|\mathcal{D}(t - 3600, t]| < 20, \ \ t - t_u^{\mathrm{last}} \geq 3600 \quad (15)$$

$$\mathrm{esc}_k = \mathbb{1}[\rho_k \geq \tau_x \ \wedge \ 0 \leq t_k - t_u^c \leq 1800] \quad (16)$$

The lock is reversible and lifts automatically after two hours unless an analyst keeps it, which leaves time for a decision without letting a false positive lock a user out for a working day. Other disruptive actions (credential reset, host isolation, network block) always wait for approval, and every action, approval and reversal is written to an audit store (T4) that also restores active locks after a restart.

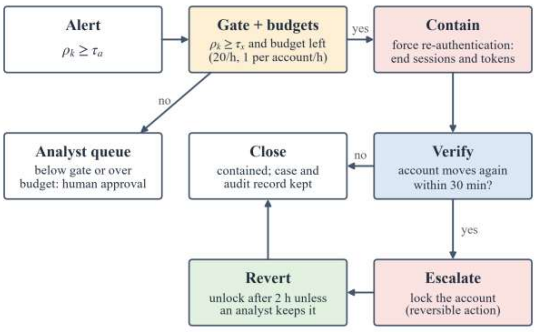


Fig. 3. Bounded response loop. Autonomous containment requires the execution gate and free budget; continued movement within 30 min escalates to a reversible two-hour account lock.

F. Grounded Report Agent

Triage notes and incident reports are generated by a LangGraph workflow over a locally hosted model (qwen3.5:4b served by Ollama), so no log data leaves the host. The workflow has four steps: prepare assembles an evidence bundle with identifiers (E-nnn for alert evidence, F-nnn for incident facts); draft asks the model for a structured report; validate rejects any statement that cites a missing identifier, names an alert outside the incident, claims an action the system did not take or raises the severity (T5); and repair returns the specific violation to the model. After three failed attempts, or if the model is unavailable, a deterministic writer produces the report from the same bundle, and every response states which path produced it. The prompt also requires the first step recommended to the analyst to be a verification rather than an approval to isolate or block. Fig. 4 reproduces part of one report.

Critical account C2287$@DOM1 authenticated across 4 hosts with high fused risk

Incident INC-C2287$@DOM1-2932909 · severity CRITICAL · window 93 s · 5 alerts · hosts C5554, C1137, C988, C4483

Summary. Account C2287$@DOM1 authenticated from four distinct hosts within a 93-second window, reaching the highest fused risk of 0.8464. [F-001, F-006, F-007]

What the system did automatically. The system ran force_reauth on C2287$@DOM1 with outcome dry_run and authority plan, followed by lock_account with outcome dry_run and authority escalation. [F-008, F-009] […]

Uncertainty. The account qualified for an unattended action again 24s after the first force_reauth attempt, requiring escalation to lock_account. [F-019]

Fig. 4. Verbatim excerpt ([…] marks omissions) of a report drafted by the local model on the demonstration stream (synthetic events over LANL identities). Cited identifiers must exist in the evidence bundle.

VI. RESULTS AND DISCUSSION

A. Experimental Setup and Metrics

Thresholds are chosen on validation data only and each test partition is scored once. Precision and recall (17) are computed from true positives (TP), false positives (FP) and false negatives (FN). Because attacks are rare, the primary metric is PR-AUC [26], computed as average precision over the ranked events (18); ROC-AUC and the false-positive rate FP/10k = $10^4 \cdot \mathrm{FP}/N_b$ are reported alongside it. Intervals are 95% stratified bootstrap intervals [27] with 2,000 (C1) or 1,000 (C2) resamples, and detectors are compared on identical

resamples. The baselines, scored on the same partitions, are a hand-written rule, a rarity score, an isolation forest [28] and a class-weighted logistic regression over the 27 features.

$$P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \quad R = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}} \tag{17}$$

$$\mathrm{AP} = \sum_{n} (R_n - R_{n-1}) P_n \tag{18}$$

B. Detection on LANL

Table II reports test PR-AUC. On C1 the TGN reaches 0.821 [0.764, 0.878] with ROC-AUC 0.999, significantly above the shipped noisy-OR pipeline (0.805; paired difference +0.016 [+0.006, +0.031]) and far above the logistic baseline (0.246). At its validation-derived threshold the noisy-OR pipeline recalls 90.5% of test attacks at 34.8 FP/10k.

TABLE II. TEST PR-AUC ON THE TWO LANL CORPORA

Detector	C1 (126 attacks)	C2 (101 attacks)
Chance (prevalence)	0.001	0.004
Hand-written rule	0.071	0.548
Rarity score	0.007	0.288
Isolation forest	0.041	0.244
Logistic regression	0.246	**0.956**
Linear fusion (TGN + channels)	0.577 [0.498, 0.656]	0.676 [0.588, 0.757]
Noisy-OR fusion (shipped)	0.805 [0.741, 0.867]	0.604 [0.520, 0.696]
TGN probability	**0.821** [0.764, 0.878]	0.566 [0.480, 0.660]
Best detector, dominant attacker host held out	0.0019	0.0003

Two checks change how these numbers should be read. First, 121 of the 126 test attacks come from one host that never appears in benign training traffic. Holding its attacks out and scoring the remaining five against all benign events lowers the TGN from 0.821 to 0.0019 [0.0017, 0.0037], so 99.8% of the headline figure rests on one campaign. Second, on C2 the ordering reverses: the logistic regression reaches 0.956 (ROC-AUC 0.999), whereas the TGN reaches 0.566 [0.480, 0.660] and its fused variants 0.604 and 0.676, and the pipeline's promotion gate, which requires a candidate to beat the best baseline on validation, rejected the TGN. Holding out the dominant host on C2 lowers the best fused detector from 0.676 to 0.0003.

The reversal is a warning rather than a contradiction. The corpora differ in local logons, sampling stride and time span, and each test partition is dominated by one attacker; which model wins depends on these choices, consistent with the re-evaluations in [9] and the benchmark critiques in [16], [17]. We therefore read the LANL scores as comparisons under a stated protocol, not as field accuracy, and we do not claim that the TGN is better than simpler models.

C. Ablation: What the Memory Contributes

Whether the TGN learned the dominant campaign's behaviour or merely its host was tested with a score-time ablation on the C1 test partition (Table III): each condition replaces part of the scorer's input while the memory trajectory stays identical. Zeroing every memory lowers PR-AUC from 0.821 to 0.389, mostly because benign scoring collapses (34.5 to 969.8 FP/10k) as every ordinary event starts to look unfamiliar; recall on the dominant host's attacks at the alert threshold falls only from 94.2% to 81.8%, and neutralising the five entity-degree features still leaves 78.5%. The campaign is thus recognised from behaviour spread across features rather than from an identity channel, and the memory mainly serves to recognise benign familiarity; the model has, however, seen only one campaign of this kind.

TABLE III. SCORE-TIME ABLATION ON THE C1 TEST PARTITION

Condition	PR-AUC	Main-host recall	FP/10k
Full model	0.821	94.2%	34.5
Destination memory zeroed	0.469	90.9%	719.2
All memory zeroed	0.389	81.8%	969.8
Source-host degree at median	0.660	95.0%	55.9
Five degree features at median	0.671	78.5%	16.0

D. Prevention

A recorded corpus cannot measure prevention, because nothing in it is counterfactual. We therefore replay campaigns of known length through the production composition of the model (features, TGN with restored memory, channels, chain rule, fusion, thresholds and response planner). Benign traffic is resampled from each user's own history, and the attacker operates from an ordinary busy workstation that also carries benign activity, so host identity cannot separate the classes. The grid has two families, a chain that carries one credential from host to host and a fan-out that rotates harvested credentials from one host, at ten inter-hop intervals from 15 s to 1 h, with five replicates of eight hops: 100 campaigns and 800 attack events among 102,476 benign events over three simulated days, after a warm-up that restores state through the validation boundary. The prevented fraction η counts the hops that follow the first autonomous action (19), where $\mathcal{A}$ is the set of the $N = 100$ campaigns that received one, a_c is the index of the triggering hop and $L = 8$.

$$\eta = \frac{1}{NL} \sum_{c \in \mathcal{A}} (L - a_c - 1) \tag{19}$$

TABLE IV. PREVENTION INSTRUMENT RESULTS

Measure	Chain	Fan-out	All
Campaigns (8 hops each)	50	50	100
Detected (alert raised)	36	9	45
Median first-alert hop index	4	5	4
Autonomous action taken	36	2	38
Attack hops prevented	27.8%	1.5%	14.6%
Benign alerts per 10k	30.8 (316 of 102,476 events)		
Automatic actions per 10k	5.5 (56 events, 56 users)		

Table IV summarises the outcome. The model alerts on 45 of the 100 campaigns with a median detection latency of four hops, the hop-count counterpart of time to detect, and acts without approval on 38, preventing 14.6% of all attack hops. The benefit lies in chains: all 36 detected chains are acted on, 33 at the fifth hop and three at the fourth, preventing 27.8% of chain hops, whereas only 2 of 50 fan-outs are acted on (1.5%). Because the rule needs four novel hops of evidence, at most half of an eight-hop chain can be prevented. Fig. 5(a) shows the boundary an attacker must cross to evade: every chain is caught when its hops are at most 10 min apart and almost none when they are 15 min or more apart, because four hops then no longer fit the 1,800-s window. Each of the 5.5 automatic actions per 10,000 benign events is a session reset that the user resolves by signing in again.

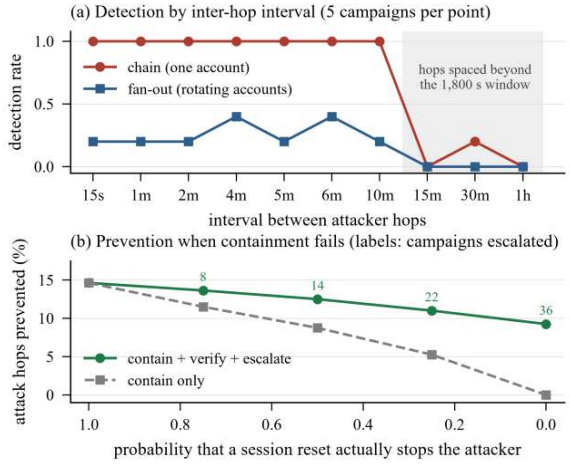


Fig. 5. Prevention instrument (100 campaigns). (a) Detection rate by the interval between hops; (b) share of attack hops prevented when a session reset stops the attacker only with the given probability.

Session resets do not always stop an attacker; Kerberos tickets that were already issued, for instance, stay valid. Fig. 5(b) varies the probability that a reset stops the campaign. Without verification and escalation, prevention falls linearly to zero; with them, the loop notices continued movement and locks the account, retaining 9.3% prevention when resets never work (36 campaigns escalated). The instrument is only meaningful for a model that has not seen its generator: when 60 campaigns from the same generator were added to training, the retrained model detected 94 of the 100 instrument campaigns at the first or second hop, but its PR-AUC on the real C1 test partition fell from 0.805 to 0.403 [0.318, 0.491], so the shipped model excludes such data.

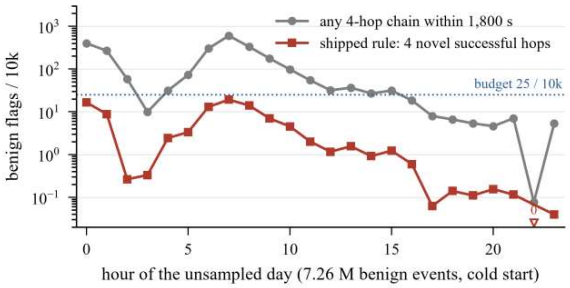


Fig. 6. Benign cost of two chain rules, hour by hour, on an unsampled LANL day processed from a cold start. Open triangles mark hours without a single flag.

E. Cost of the Chain Rule at Full Rate

Rule costs measured on a stride-sampled corpus are misleading, because keeping one benign event in several hundred removes nearly every benign multi-hop pattern. On C1 a four-hop, 300-s chain rule appeared to cost 4.6 FP/10k; on an unsampled day of LANL (7,263,653 events after local logons are removed) the same rule flagged 715 to 855 benign events per 10,000. Fig. 6 compares, hour by hour on that day and from a cold start, a rule that flags any four-hop chain within 1,800 s with the shipped rule. Requiring successful hops into never-reached hosts lowers the day's cost from 114.4 to 4.3 per 10,000, a 26-fold reduction, and the cost keeps falling as history accumulates, to about 0.1 per 10,000 in the last six hours. No chain campaign is lost, since a walk into new hosts satisfies both tests at the same hop.

F. External Test on OTRF

The frozen model (LANL-trained checkpoint and thresholds, no retraining) scored the 21 OTRF recordings through its own ingestion path, which recognised every recording as Windows Security JSON. Table V and Fig. 7 give the result. At the shipped threshold only 2 of 21 attacks are detected, a PurpleSharp playbook (risk 0.961) and an Empire WMIC backdoor (0.734), which are also the busiest recordings and the source of the 13 benign alerts. Without a threshold the signal is real but weak: the pooled ROC-AUC is 0.661 [0.564, 0.754] under a bootstrap over recordings, the mean within-recording ROC-AUC is 0.787, and the attacker's event is ranked first in 10 of 18 recordings against 6.3 expected by chance.

TABLE V. EXTERNAL TEST ON 21 OTRF RECORDINGS

Measure	Value
Attack / benign events	77 / 118
Recordings detected at $\tau_a = 0.329$	2 of 21
Benign events alerted at τ_a	13 of 118
Attack ranked first (expected by chance)	10 of 18 (6.3)
Pooled ROC-AUC [95% CI]	0.661 [0.564, 0.754]
Mean within-recording ROC-AUC	0.787 (18 recordings)

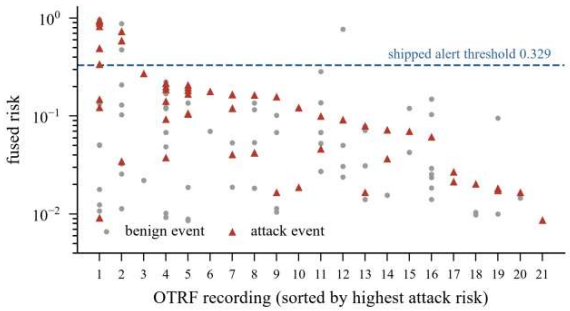


Fig. 7. Fused risk of every event in the 21 OTRF recordings, scored by the frozen LANL model. Only two attacks exceed the alert threshold, although attacks usually outrank the benign events of their own recording.

Three known effects combine here. Every OTRF identity is outside the frozen dictionary, so the model reasons from shared out-of-vocabulary memory; every recording holds a single hop, so the chain rule cannot fire by construction; and a threshold is a property of a score distribution, which moves between networks. Reading a new estate's logs therefore works, but alerting correctly on them requires warm-up on that estate's own history and a re-derived gate.

G. Threats to Validity

The LANL labels describe one red-team exercise with four attacker hosts, one of which produced 94% of the events, so no absolute detection figure here should be read as performance against a new attacker. Both LANL corpora are stride-sampled, which inflates prevalence and, because features are computed on the sampled stream, shortens benign histories relative to attack histories; Section VI-E shows how large such effects are for rules, and repeating the model comparison with full-rate features is our first item of future work. Prevention is measured on simulated campaigns whose generator is independent of training but is still a model of attacker behaviour, and the OTRF recordings are short and come from one lab.

VII. CONCLUSION AND FUTURE WORK

GraphSentinel shows that a temporal graph neural network can sit inside a complete and bounded model for detecting and preventing credential-based lateral movement: it reads heterogeneous logs, scores every logon from past-only features, fuses evidence auditably, acts alone only above a precision bound, contains, verifies, escalates and reverts within budgets, and explains itself through grounded reports. Measured end to end, the loop prevents 14.6% of attack hops in simulated campaigns (27.8% for credential chains) at 5.5 automatic actions per 10,000 benign logons, and escalation preserves prevention when containment fails.

The evaluation is a result in its own right: the headline LANL score depends on one attacker host, a logistic regression outperforms the TGN on a corrected corpus, sampled corpora misstate rule costs by two orders of magnitude, and a frozen model transfers its ranking but not its threshold to an unseen network. Future work will (i) recompute features at full rate and repeat the model comparison with gradient-boosted trees and several seeds, (ii) evaluate on further labelled corpora with held-out attacker hosts, (iii) adapt thresholds to each network from a short warm-up period, and (iv) measure the accuracy of the report agent with practising analysts.

REFERENCES

[1] MITRE ATT&CK, "Lateral Movement, Tactic TA0008 – Enterprise," The MITRE Corporation. [Online]. Available: https://attack.mitre.org/tactics/TA0008/ (accessed Sep. 2026).

[2] H. Siadati and N. Memon, "Detecting structurally anomalous logins within enterprise networks," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, Dallas, TX, USA, 2017, pp. 1273–1284.

[3] G. Ho, M. Dhiman, D. Akhawe, V. Paxson, S. Savage, G. M. Voelker, and D. Wagner, "Hopper: Modeling and detecting lateral movement," in *Proc. 30th USENIX Security Symp.*, 2021, pp. 3093–3110.

[4] F. Liu, Y. Wen, D. Zhang, X. Jiang, X. Xing, and D. Meng, "Log2vec: A heterogeneous graph embedding based approach for detecting cyber threats within enterprise," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, London, U.K., 2019, pp. 1777–1794.

[5] B. Bowman, C. Laprade, Y. Ji, and H. H. Huang, "Detecting lateral movement in enterprise computer networks with unsupervised graph AI," in *Proc. 23rd Int. Symp. Research in Attacks, Intrusions and Defenses (RAID)*, 2020, pp. 257–268.

[6] I. J. King and H. H. Huang, "Euler: Detecting network lateral movement via scalable temporal link prediction," in *Proc. Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, 2022.

[7] J. Khoury, Đ. Klisura, H. Zanddizari, G. De La Torre Parra, P. Najafirad, and E. Bou-Harb, "Jbeil: Temporal graph-based inductive learning to infer lateral movement in evolving enterprise networks," in *Proc. IEEE Symp. Security and Privacy (SP)*, San Francisco, CA, USA, 2024, pp. 3644–3660.

[8] A. D. Kent, "Comprehensive, multi-source cyber-security events," Los Alamos National Laboratory, 2015, doi: 10.17021/1179829.

[9] C. Larroche, "On fair and realistic performance evaluations for graph-based lateral movement detectors," arXiv:2607.29390, Jul. 2026.

[10] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Security and Privacy*, Oakland, CA, USA, 2010, pp. 305–316.

[11] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symp.*, Boston, MA, USA, 2022, pp. 3971–3988.

[12] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, "Temporal graph networks for deep learning on dynamic graphs," arXiv:2006.10637, 2020.

[13] S. Kumar, X. Zhang, and J. Leskovec, "Predicting dynamic embedding trajectory in temporal interaction networks," in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, Anchorage, AK, USA, 2019, pp. 1269–1278.

[14] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, and K. Achan, "Inductive representation learning on temporal graphs," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2020.

[15] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Doha, Qatar, 2014, pp. 1724–1734.

[16] F. Poursafaei, S. Huang, K. Pelrine, and R. Rabbany, "Towards better evaluation for dynamic link prediction," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, Datasets and Benchmarks Track, 2022.

[17] W. Cong, S. Zhang, J. Kang, B. Yuan, H. Wu, X. Zhou, H. Tong, and M. Mahdavi, "Do we really need complicated model architectures for temporal networks?" in *Proc. Int. Conf. Learning Representations (ICLR)*, Kigali, Rwanda, 2023.

[18] A. Nelson, S. Rekhi, K. Scarfone, and M. Souppaya, "Incident response recommendations and considerations for cybersecurity risk management: A CSF 2.0 community profile," NIST Special Publication 800-61r3, Apr. 2025, doi: 10.6028/NIST.SP.800-61r3.

[19] Z. Inayat, A. Gani, N. B. Anuar, M. K. Khan, and S. Anwar, "Intrusion response systems: Foundations, design, and challenges," *J. Network and Computer Applications*, vol. 62, pp. 53–74, 2016.

[20] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, art. 248, 2023.

[21] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," NIST Special Publication 800-207, Aug. 2020, doi: 10.6028/NIST.SP.800-207.

[22] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proc. IEEE*, vol. 63, no. 9, pp. 1278–1308, Sep. 1975.

[23] R. Rodriguez and J. L. Rodriguez, "Security Datasets," Open Threat Research Forge (OTRF). [Online]. Available: https://github.com/OTRF/Security-Datasets (accessed Sep. 2026).

[24] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. San Mateo, CA, USA: Morgan Kaufmann, 1988.

[25] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[26] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.

[27] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

[28] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422.